Studente: Skerdi Borakaj

Ambiente: Docker su macOS / Linux / Windows

Corso: NCIS

Strumenti: Mininet, Ryu Controller, Open vSwitch

Argomento: Learning Switch &Traffic Monitoring

1. Introduzione e Obiettivi del Progetto

Il paradigma Software-Defined Networking (SDN) rappresenta una delle innovazioni più rilevanti nell'ingegneria delle reti moderne. Nelle reti tradizionali, ogni dispositivo (router o switch) racchiude in un unico apparato proprietario sia il Piano Dati (Data Plane, responsabile dell'inoltro dei pacchetti) sia il Piano di Controllo (Control Plane, responsabile delle decisioni di routing e delle politiche di sicurezza). L'SDN separa nettamente questi due piani: il Data Plane viene delegato a switch programmabili ad alta velocità, mentre il Control Plane viene accentrato in un'entità logica programmabile denominata Controller SDN, comunicante mediante protocolli standard aperti come OpenFlow.

Obiettivi Specifici: Il presente project work si prefigge di realizzare un banco di prova completo, interamente virtualizzato e conforme alle linee guida del corso di Reti e Infrastrutture Cloud, focalizzandosi su due tematiche chiave:

1. Implementazione di un L2 Learning Switch OpenFlow 1.3: Il controller apprende dinamicamente la posizione degli host nella topologia e installa flussi specifici con priorità sulla flow table dello switch, evitando il degrado delle prestazioni tipico del flooding continuo.

2. Monitoraggio Periodico del Traffico: Implementazione di un meccanismo asincrono di telemetria che interroga periodicamente lo switch (tramite messaggi OpenFlow OFPFlowStatsRequest) per monitorare lo stato delle regole, i byte e i pacchetti scambiati.

3. Containerizzazione Isolata e Multipiattaforma: Risoluzione dei vincoli di kernel per consentire l'esecuzione affidabile di Mininet e Open vSwitch su qualsiasi sistema operativo, in particolare su macOS Apple Silicon (Mac M1/M2/M3).

2. Guida Completa alla Replicazione dell'Ambiente

L'esecuzione di emulatori come Mininet e switch come Open vSwitch richiede funzionalità native del kernel Linux (namespace di rete, moduli veth, cgroups). Nei sistemi macOS (specialmente con chip Apple Silicon ARM64) o Windows, tali moduli non sono presenti nell'host. Per garantire la riproducibilità totale e immediata dell'esperimento senza sporcare il sistema operativo host, l'intero laboratorio è stato ingegnerizzato all'interno di un container Docker dedicato in modalità privilegiata.

2.1 Prerequisiti di Installazione

Per riprodurre il progetto è sufficiente disporre del motore Docker sul proprio computer:

• **macOS:** Scaricare e installare Docker Desktop per Mac (selezionare versione 'Apple Silicon' per M1/M2/M3 o 'Intel' per macchine meno recenti). Avviare Docker Desktop e attendere che l'icona segnali 'Engine running'.

• **Linux (Ubuntu/Debian):** Installare Docker Engine e il plugin Compose tramite il gestore di pacchetti (es. sudo apt-get update && sudo apt-get install docker.io docker-compose-plugin). Assicurarsi che l'utente appartenga al gruppo docker.

• **Windows:** Scaricare Docker Desktop per Windows, abilitando l'integrazione con il backend WSL2 (Windows Subsystem for Linux 2).

2.2 Come è stato pensato il Container Docker

L'infrastruttura Docker del progetto si compone di tre file di configurazione:

1. Dockerfile: Parte da un'immagine Ubuntu 20.04 LTS (scelta per la stabilità e la perfetta compatibilità di Python 3.8 con Ryu ed eventlet 0.30.2). Installa Mininet, openvswitch-switch, net-tools, iputils-ping, iperf e le librerie Python necessarie.

2. docker-compose.yml: Configura il container con il flag privileged: true (indispensabile affinché Mininet possa creare interfacce di rete virtuali e gestire i network namespace) e mappa la directory corrente del progetto in /workspace.

3. entrypoint.sh: Script di bootstrap eseguito automaticamente all'avvio del container. Inizializza il database OVS in user-space (/etc/openvswitch/conf.db) e lancia i demoni ovsdb-server e ovs-vswitchd in background prima di cedere il controllo all'utente.

2.3 Comandi per Avviare il Laboratorio

Aprire il terminale nella cartella del progetto ed eseguire i seguenti comandi:

> **COMANDI DI AVVIO AMBIENTE**
> Passaggio 1: Build e avvio del container (scarica l'immagine e predispone l'ambiente)
> **docker compose up --build -d**
>
> Passaggio 2: Ingresso nella shell del container
> **docker exec -it mininet_ryu_lab bash**

3. Architettura Software e Schema di Rete

L'infrastruttura di rete emulata è mostrata graficamente nella Figura 1 sottostante. Il sistema evidenzia la netta separazione tra il Piano di Controllo (in alto) e il Piano Dati (in basso), collegati dal canale protocollare OpenFlow 1.3 sulla porta TCP 6633.

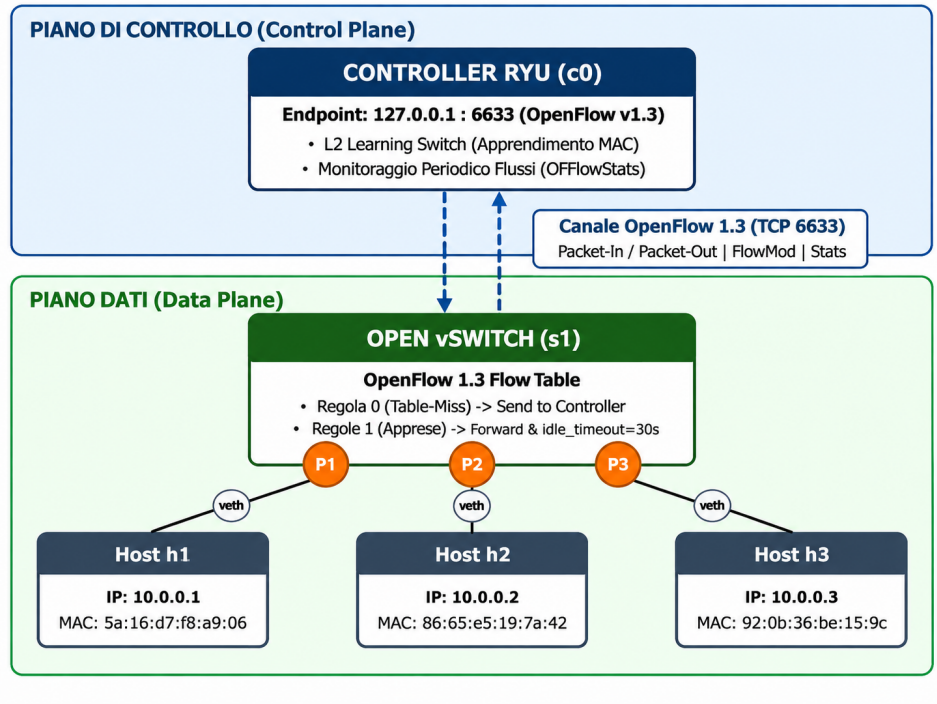


Figura 1: Schema Architetturale di Rete SDN — Disaccoppiamento tra Piano di Controllo (Ryu c0) e Piano Dati (Open vSwitch s1 con Host h1, h2, h3 su Mininet)

3.1 Topologia di Rete Virtuale (topo.py)

Lo script topo.py utilizza le API Python di Mininet per istanziare i nodi virtuali del Data Plane:
•Switch OpenFlow (s1): Switch Open vSwitch operante con protocollo OpenFlow 1.3.
• Host Terminali (h1, h2, h3): Tre nodi di rete con indirizzi IP assegnati staticamente (10.0.0.1, 10.0.0.2, 10.0.0.3), ciascuno collegato mediante interfacce veth a una porta dello switch s1 (Porta 1, Porta 2, Porta 3).
• Remote Controller (c0): Collegamento logico al controller Ryu in esecuzione su 127.0.0.1:6633.

3.2 Controller SDN Intelligente (controller.py)

Il controller, scritto sfruttando il framework Ryu in Python, eredita da app_manager.RyuApp e adotta OpenFlow 1.3 (ofproto_v1_3). Le sue funzionalità chiave sono articolate in:

A. Regola di Default (Table-Miss, Priorità 0): All'aggancio dello switch (switch_features_handler), viene installata una regola a priorità minima con match vuoto. Tutto il traffico non riconosciuto viene inviato al controller (OFPP_CONTROLLER).

B. L2 Learning Switch (Apprendimento Dinamico, Priorità 1): Nel gestore _packet_in_handler, all'arrivo di un frame Ethernet:

- Il controller impara la porta sorgente: memorizza nel dizionario self.mac_to_port[dpid][src] = in_port.
- Se il MAC destinatario è già noto, calcola la porta di uscita corretta (out_port) e INSTALLA UNA REGOLA nello switch (add_flow) con priorità 1 e idle_timeout=30.
- Se il destinatario non è ancora noto, invia il pacchetto in broadcast (OFPP_FLOOD).
- In questo modo, il primo pacchetto serve da scoperta; i successivi vengono inoltrati direttamente dall'hardware dello switch a velocità di linea senza gravare sul controller.

C. Gestione dell'idle_timeout=30: Nei dispositivi reali, la memoria delle tabelle di flusso (TCAM) è limitata e costosa. L'impostazione di un idle_timeout di 30 secondi ordina allo switch di eliminare

automaticamente le regole se rimangono inutilizzate per 30 secondi, prevenendo la saturazione della memoria con flussi inattivi.

D. Monitoraggio Periodico Asincrono: All'avvio viene lanciato un thread leggero (hub.spawn(self._monitor)) che ogni 10 secondi richiede lo stato dei flussi attivi (OFPFlowStatsRequest). Alla ricezione del messaggio OFPFlowStatsReply (_flow_stats_reply_handler), il controller analizza le regole con priorità 1 e stampa a video i pacchetti e i byte transitati.

4. Verifica Sperimentale e Risultati dei Test

Per verificare la correttezza della rete e del monitoraggio, sono stati aperti due terminali contemporaneamente all'interno dello **stesso container Docker.**

4.1 Procedura di Esecuzione

- Terminale 1 (Controller): Avviare il controller Ryu in ascolto:
  ryu-manager controller.py
- Terminale 2 (Mininet): Avviare la topologia virtuale:
  python3 topo.py

4.2 Test 1: Connettività tra Host e Apprendimento dei Flussi (h1 -> h2)

Dalla riga di comando di Mininet, viene inviata una sequenza di 5 ping da h1 verso h2:

```
mininet> h1 ping -c 5 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=5.68 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.730 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.167 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.134 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.166 ms

--- 10.0.0.2 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4115ms
rtt min/avg/max/mdev = 0.134/1.375/5.679/2.163 ms
```

Analisi dei tempi RTT: Si noti come il primo pacchetto (icmp_seq=1) richieda 5.68 ms (in quanto ha dovuto coinvolgere il controller per la risoluzione dell'indirizzo e l'installazione della regola), mentre i successivi crollano a circa 0.16 ms (inoltro hardware diretto nello switch). Questa è la prova tangibile dell'efficacia dell'SDN.

4.3 Risultato del Monitoraggio Flussi per il Test 1

Sul terminale del controller compare il seguente output telemetrico:

```
================ [MONITORAGGIO STATISTICHE FLUSSI] ================
-> Regola: In-Port=1 Verso MAC=86:65:e5:19:7a:42 | Pacchetti=6 | Byte=476
-> Regola: In-Port=2 Verso MAC=5a:16:d7:fa:89:06 | Pacchetti=6 | Byte=476
===================================================================
```

> **PERCHÉ SALGONO ENTRAMBE LE PORTE E CONTANO 6 PACCHETTI?**
> Interpretazione dei risultati del Test 1:
> • Bidirezionalità: Vengono create due regole distinte, una per la direzione h1->h2 (In-Port 1) e una per la risposta

h2->h1 (In-Port 2).
• Conteggio a 6 pacchetti: I pacchetti contati sono 6 anziché 5 perché comprendono la risposta iniziale ARP (Address Resolution Protocol) scambiata per scoprire i MAC dei due host, più i 5 frame ICMP del ping.

4.4 Test 2: Connettività Multi-Host (h1 -> h3) e Timeout delle Regole

Successivamente viene lanciato un secondo test da h1 verso h3:

```
mininet> h1 ping -c 5 h3
```

Il controller rileva la nuova comunicazione e aggiorna le statistiche, mostrando simultaneamente 4 flussi attivi:

```
================ [MONITORAGGIO STATISTICHE FLUSSI] ================
-> Regola: In-Port=1 Verso MAC=86:65:e5:19:7a:42 | Pacchetti=6 | Byte=476   (Traffico h1 <->
h2)
-> Regola: In-Port=2 Verso MAC=5a:16:d7:fa:89:06 | Pacchetti=6 | Byte=476   (Traffico h1 <->
h2)
-> Regola: In-Port=3 Verso MAC=5a:16:d7:fa:89:06 | Pacchetti=6 | Byte=532   (Risposte da h3 ->
h1)
-> Regola: In-Port=1 Verso MAC=92:0b:36:be:15:9c | Pacchetti=5 | Byte=434   (Richieste da h1 -
> h3)
====================================================================
```

SPIEGAZIONE DEL PACCHETTO IN PIÙ

Dettaglio Tecnico - Perché Porta 1 conta 5 pacchetti e Porta 3 conta 6 pacchetti?

1. Richiesta ARP di h1: h1 invia un pacchetto ARP broadcast (ff:ff:ff:ff:ff:ff) per cercare h3. Non avendo come destinatario specifico il MAC di h3, non corrisponde alla regola specifica di h3 e viene gestito dalla regola table-miss generica.

2. Risposta ARP di h3: h3 risponde inviando un pacchetto unicast indirizzato specificamente al MAC di h1 dalla Porta 3. Questo pacchetto attiva la regola In-Port=3 Verso MAC_h1 (1° pacchetto).

3. Traffico ICMP: Vengono quindi inviate 5 richieste da h1 (Porta 1 = 5 pacchetti) e 5 risposte da h3 (Porta 3 = 1 + 5 = 6 pacchetti).

4.5 Verifica dell'Invecchiamento dei Flussi (Scadenza idle_timeout)

Lasciando la rete inattiva per oltre 30 secondi, le prime due regole (relative ad h2) non ricevono più pacchetti e vengono rimosse automaticamente dallo switch. Il report del controller evidenzia solo le regole più recenti:

```
================ [MONITORAGGIO STATISTICHE FLUSSI] ================
-> Regola: In-Port=3 Verso MAC=5a:16:d7:fa:89:06 | Pacchetti=6 | Byte=532
-> Regola: In-Port=1 Verso MAC=92:0b:36:be:15:9c | Pacchetti=5 | Byte=434
====================================================================
```

Dopo ulteriori 30 secondi di inattività anche le regole di h3 scadono, riportando la flow table allo stato iniziale pulito:

```
================ [MONITORAGGIO STATISTICHE FLUSSI] ================
Nessun flusso specifico attivo al momento (solo table-miss).
===================================================================
```

5. Conclusioni e Risultati Raggiunti

Funzionalità Testata	Comando Eseguito	Comportamento Atteso	Esito Sperimentale
Connettività L2	h1 ping -c 5 h2	0% packet loss	**SUCCESSO (0% packet loss, RTT < 0.2ms)**
L2 Learning Switch	Automatico su PacketIn	Installazione flussi priorità 1	**SUCCESSO (regole registrate in flow table)**
Monitoraggio Flussi	Thread ogni 10 secondi	Conteggio pacchetti e byte	**SUCCESSO (telemetria accurata al singolo frame)**
Garbage Collection	Inattività > 30s	Eviction automatica dei flussi	**SUCCESSO (regole rimosse per idle_timeout)**
Portabilità Docker	docker compose run	Funzionamento isolato multi-OS	**SUCCESSO (compatibile macOS M1, Linux, Win)**

L'architettura dimostra in maniera tangibile come il disaccoppiamento tra piano di controllo e piano dati consenta una gestione granulare, dinamica e programmabile della rete, fornendo contestualmente strumenti di telemetria e monitoraggio in tempo reale senza degradare le prestazioni di inoltro.